

## 711 (1) Synchronising logical clock

$a \rightarrow b$

$a$  occurs before  $b$ .

Logical clock TS  $(\underbrace{C(a)}_{a \text{ 的时间值}})$

$a \rightarrow b$

$$C(a) < C(b).$$

total order.  $(T_a, I_d a)$

$$\begin{cases} a \rightarrow b & \text{if } T_a < T_b \\ a \rightarrow b & \text{if } T_a = T_b, I_d a < I_d b \end{cases}$$

## 711 (2)

partially ordered logical clock

$b \{ (a, 1), (b, 1) \}$

子进程继承父进程的时间戳，再加上自己的TS.

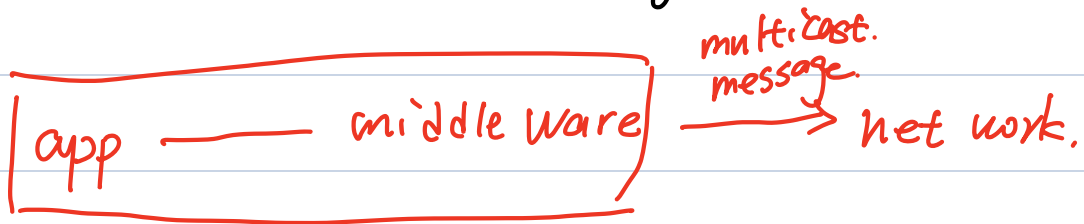
交换信息时相互影响，互相继承时间戳，同 process 的 TS 更新为最大的 inherit.

先写没被影响的 TS，再考虑影响.

more expressive.

impractical expensive.  $\rightarrow$  long-live computer.

Multicast. aim: each process receive a copy of message which sent to group.



FIFO ordering: a process send  $m$  before  $m'$   
other receive  $m$  before  $m'$

发送顺序和接收一致

→ causal ordering 的区别: one process send several message.

711 (3)

Causal ordering can different process send several message

$m$  happen before  $m'$

other should receive  $m$  before  $m'$

ordered multicast.

Total ordering: a process send  $m$  before  $m'$

all ....

## 🧠 和 FIFO / Causal 的区别

| 类型              | 保证顺序的范围                | 关注点         | 是否能看出发送者是谁 |
|-----------------|------------------------|-------------|------------|
| FIFO Ordering   | 每个发送者自己的消息按顺序被 deliver | 同一个发送者的消息顺序 | ✅ 是谁发的很重要  |
| Causal Ordering | 所有消息间的因果关系             | 谁“触发”谁，先因后果 | ✅ 谁触发了谁很关键 |
| Total Ordering  | 所有消息必须在所有进程中顺序完全一致     | 全局一致的消息顺序   | ❌ 谁发的不重要   |

### 🔄 举例对比（假设有两个发送者 P1 和 P2）

#### 1. FIFO

- 如果 P1 发了 m，接着发了 m'
- 所有进程必须先 deliver m，再 deliver m'（只管 P1 发的）

#### 2. Causal

- P1 发了 m，P2 收到后根据 m 发了 m'
- 所有进程必须先 deliver m，再 deliver m'（因为有因果关系）

#### 3. Total

- 不管消息 m 和 m' 是否有因果关系、是否来自同一发送者
- 只要有一个进程先 deliver 了 m，再 deliver m'
- 所有进程都必须保持一样的顺序

### ✅ 总结一句话：

- FIFO ordering: 同一发送者发的消息顺序要一致。
- Causal ordering: 有先因后果的消息顺序要一致。  
↓
- Total ordering: 所有消息的顺序在所有人眼中都要一致，哪怕它们无关。

## Reliable multicast

send to all the process  
or send to none of them

每个接收者都会发送已收到的消息到其他  
进程 (除了 sender)

complexity  $n + n-1 + n-2 + \dots + 1 = n^2$

## Virtual Synchrony

multicast group

进程 加入 或 退出 组,

view

view changed  
message sent  
to all process

Virtual Synchrony only ensure

the sequence of view changed message

无视 crash 的, 保证 reliable multicast

每个视图里的消息要一致才能保证 View-S...

change the view together

implement of FIFO ordering

use sequence number.

sender send message, Sequence number + 1

receiver, if message sequence number

= receiver sequence + 1

deliver, else in

wait queue.

implement of causal ordering

vector variable.

$$V_i[1] = 0$$



variable  
in process i

message from process i

消息变量

① 发送时自增, 更新

更新 process

vector variable.

② deliver 后更新

process vector variable

rule: 比较消息的 variable.

保证是该进程  
的下一条消息,

process 的 variable.

若等于消息的 variable  
+1,

保证其他进程的

$V_j[k] \leq V_i[k]$  deliver

所有相关消息已收到

否则 in wait queue

implement total order.

send message with TS

wait all receiver. back TS

choose largest TS send to receiver

update TS process as final TS

deliver rule

① final TS

② first message  
in que.

# Mutual Exclusion

only one task is allowed to access the shared resource.

## centralized approach

draw back.

access manager — single point of failure.

## requirement

① Safety — one task can access

② Liveness — no endless wait

③ Fairness — fair chance to access

Timestamp - based. Ricart and Agrawala

algorithm.  
each request give Timestamp

Total order

who need resource would send request

to others with TS, who receive all  
ok from other process

smallest TS programmer would  
access to resource. order  
↓  
liveness

the value of the TS increase  
monotonically, every process have  
chance to enter critical region.  
fairness

Quorum — based.

Maekawa's algorithm.

subset of task

every quorum have intersection  
交集.

assign the process with at least one

Quorum, ask for vote, until gain  
all <sup>safety</sup> vote of that Quorum, acquire resource

if one of the Quorum have been voted,  
<sup>fairness</sup>  
compare the TS, transfer vote to the  
new one. avoid dead lock.

↓  
use inquire  
relinquish  
fix dead lock

Token-based Raymond algorithm.  
<sup>liveness</sup>  
Unique token. → allow to access the resource

need resource → send token\_request  
message

received message, no token → send to neighbour →  
until find token

every task has a queue to record the  
request

Initialization. ——— holder send message to

its neighbours, neighbour update the point to holder and send to other neighbour.

request: Send request to the neighbour where the direction of the point, neighbour if not the holder would add the request to queue and send the request to neighbour. until to holder.

holder send it back <sup>(token)</sup>, update holder as the former one, the current holder delete the request in queue, send the token back to the former one, also send the request later send. until the request finish

dead lock.

Echo Algorithm.

## probes

initiator send probes to its neighbours

non-initiator replicates and sends probes to its neighbour. after receiving probes.

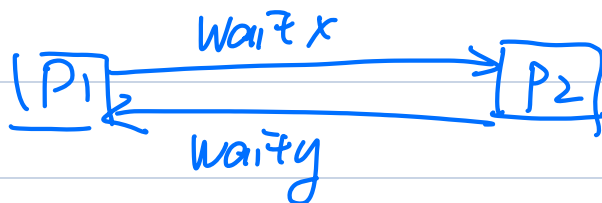
non-initiator send echo when the node have received the probes

The algorithm terminates when the initiator receives all echoes from all its neighbour.

## Dead lock detection

wait for graph.

use edge



Model of deadlock.

And model,

need all the node point it  
need all resource.  
finish.

dead lock  $\longrightarrow$  cycle

Or model need one of the node point to  
find a subset it, finished  
knot each node have at least  
one outgoing edge  
all the outgoing edge point to  
the node in subset.

Deadlock detection algorithm

① no fake deadlock

② all deadlock should be detected.

在删除 edge 前检测到新增 edge.

③ Centralized detection

use central detector.

every time change of use of  
resource would send message to

central detector, include delete edge or add edge.

because the arrival time of message different, might form a cycle (fake)

report lead lock.  
→ fake dead lock.

And model deadlock detection.

a process is waiting  
send probe  $(i, j, k)$

← 下个进程    ↓ 上个进程    ↘ 下个进程

initiator

receive probe → dead lock.

other process if has waiting

process, update probe and send to waiting

process, if not waiting, drop probe.

Or model dead lock detection

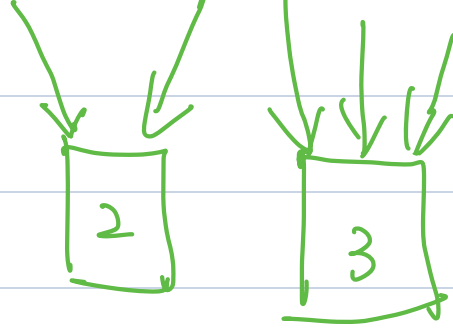
use echo algorithm.

Send probe, neighbour pass it  
to waiting ---, if not wait, ignore  
probe, if node have receive probe,  
send echo back, initiator receive  
all echo  $\rightarrow$  dead lock.

Sb.

Garbage collection

reference counting



if reference counting = 0.

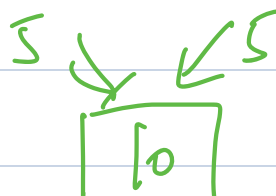
release the memory.

problem: because of message delay  
memory would be released too  
early, when cloned.

Weighted reference counting.

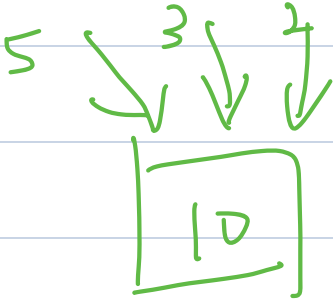
a object have a weight

assign weight to each pointer.



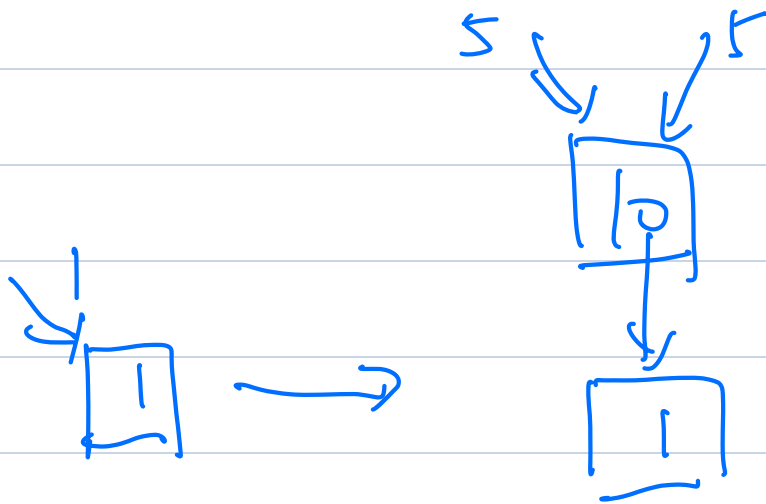
Object weight = Sum all pointer weight

copy: split the weight evenly



Weight = 0, release memory

what if can't be split?



can't handle cycle garbage.

coll